

# TSTO/00

Target State Transition Object Specification

中文名：可验证目标状态迁移对象规范

状态：Experimental Draft 00（实验草案，不是正式标准）

发布日期：2026-08-03

发起方：认知涌现（Cognitive Emergence）

许可：Creative Commons Attribution 4.0 International（CC BY 4.0）

语言地位：本中文文本为信息性翻译；发生歧义时以英文规范文本为准。

## 摘要

TSTO/00 定义一种不可变、可寻址、可验证的“目标状态迁移对象”（Target State Transition Object，简称 TSTO）。一个 TSTO 描述：某个对象基于何种可证明的初始状态，应在什么时间边界内变成什么目标状态，过程中必须满足哪些约束，以及最终依据什么验证规则判断结果是否成立。

TSTO 不描述谁承担责任，也不规定任务如何执行、价格如何形成、资金如何结算或争议如何裁决。责任事件、执行回执、市场、结算和争议由外部协议、Binding 或产品处理。

因此，TSTO 是“什么变化才算完成”的最小语义对象。它可以独立使用，也可以被责任、执行、验证和结算系统引用。

## 1. 文档状态

本文件是用于实现、试验和征求意见的 Draft 00。它可以作为原型和封闭试点的互操作依据，但不得宣称为行业共识或正式标准。

本文件中的 **MUST**、**MUST NOT**、**REQUIRED**、**SHOULD**、**SHOULD NOT**、**MAY**、**OPTIONAL** 按照 BCP 14 (RFC 2119、RFC 8174) 解释，且仅在全大写时具有规范性。

Draft 00 的冻结范围只有：

1. TSTO Core 的最小字段与语义；
2. 最小谓词语言；
3. 内容规范化与摘要计算；
4. Profile 和 Verification Policy 的约束；
5. 外部系统引用 TSTO 的通用规则。

Draft 00 不冻结行业字段、市场机制、责任分配算法或具体执行流程。

## 2. 设计目标

TSTO/00 的目标是让两个互不依赖的系统，对同一个目标变化至少能够一致回答以下问题：

- 变化针对哪个对象；
- 初始状态是什么，证据在哪里；
- 目标状态是什么；
- 何时开始、何时应达成、何时必须完成验证；
- 哪些条件在变化过程中不得被破坏；
- 使用哪套规则和证据判断结果；
- 双方引用的是否是完全相同、未被篡改的对象。

## 3. 非目标

TSTO/00 不定义以下内容：

- 执行步骤、 workflow 编排或 Agent 指令；
- 执行者、委托方或验证者身份体系；
- 授权、权限、责任上限与终止权；
- 报价、竞价、费用、托管、清分、结算和税务；
- 归因、分润、保险、担保和争议仲裁；
- 网络传输、消息队列或 API 调用方式；
- 通用状态机建模语言；
- 法律合同效力。

这些能力可以引用 TSTO，但不得被塞入 TSTO Core。

## 4. 术语

| 术语                  | 定义                                                |
|---------------------|---------------------------------------------------|
| TSTO                | 一个不可变的目标状态迁移对象。                                   |
| Subject             | 被观察并要求发生状态变化的对象。                                  |
| State Projection    | 由行业 Profile 定义、供谓词求值使用的规范化 JSON 状态视图。             |
| Baseline            | 对 Subject 初始状态的、有证据支持的断言集合。                       |
| Target              | 对期望终态的机器可求值断言集合。                                  |
| Constraint          | 状态迁移期间或终态验证时必须成立的约束。                              |
| Evidence            | 支持状态断言或验证结论的可寻址资料、事件、回执或快照。                       |
| Profile             | 将领域对象映射为 State Projection，并限制字段、类型、证据和谓词用法的版本化规范。 |
| Verification Policy | 规定证据可接受性、求值过程、时间边界和结论形成方式的不可变规则。                  |
| Resolver            | 取得并校验 TSTO、Profile、Policy 或 Evidence 的实现。         |
| Verifier            | 根据 Verification Policy 对 TSTO 作出三值判断的实现或主体。       |
| Binding             | 规定外部责任、执行、验证或结算协议如何引用 TSTO 的独立规范。                 |

## 5. 核心模型与边界

一个 TSTO Core 由九个语义部分组成：

| 部分                | 回答的问题           |
|-------------------|-----------------|
| Protocol metadata | 这是哪个版本、哪个不可变对象？ |
| Profile           | 应怎样解释该行业对象和字段？  |
| Subject           | 哪个对象要变化？        |
| Baseline          | 从什么已观察状态开始？     |
| Target            | 最终必须满足什么条件？     |
| Validity          | 在什么时间边界内成立？     |
| Constraints       | 变化不能以什么代价完成？    |
| Verification      | 用什么规则判定结果？      |
| Integrity         | 引用方看到的是否为同一内容？  |

TSTO 是静态、不可变的声明，不拥有可变 status 字段。所谓“待执行、执行中、已完成、已失败、已终止、已争议”等状态，必须由 TSTO 外部的责任事件、执行回执、时间、验证和结算记录推导，不得回写并修改原 TSTO。

## 6. 序列化与媒体类型

TSTO/00 的规范序列化是 UTF-8 JSON。

- spec 的固定值为 TSTO/00。

- type 的固定值为 TargetStateTransition。
- 提议的实验性媒体类型为 application/tsto+json，尚未注册；在完成注册前，互联网传输实现 SHOULD 同时允许 application/json。
- TSTO 可以通过 HTTP、消息队列、MCP、A2A、文件或其他方式传输；传输方式不影响其语义。
- 实现 MUST 拒绝重复 JSON 成员名。
- 实现 MUST 将未知的 Core 顶层字段视为不符合 Draft 00，而不是静默忽略。

## 7. TSTO Core 对象

### 7.1 顶层结构

```
{
  "spec": "TSTO/00",
  "id": "urn:uuid:01987654-3210-7abc-8def-0123456789ab",
  "type": "TargetStateTransition",
  "issued_at": "2026-08-02T16:00:00Z",
  "profile": {
    "id": "urn:tsto:profile:ar.invoice-settlement:v1",
    "digest": "sha-256:AAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAAA"
  },
  "subject": {
    "id": "urn:merchant:demo:invoice:INV-2026-0088",
    "type": "Invoice",
    "version": "erp-etag-184"
  },
  "baseline": {
    "observed_at": "2026-08-02T15:55:00Z",
    "claims": [
      { "op": "eq", "path": "/status", "value": "overdue" },
      { "op": "eq", "path": "/balance_due_minor", "value": 1200000 }
    ],
    "evidence": [
      {
        "id": "urn:uuid:01987654-3210-7abc-8def-111111111111",
        "kind": "erp_snapshot",
        "digest": "sha-256:BBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBBB",
        "observed_at": "2026-08-02T15:55:00Z",
        "media_type": "application/json"
      }
    ]
  },
  "target": {
    "claims": [
      { "op": "eq", "path": "/status", "value": "settled" },
      { "op": "eq", "path": "/balance_due_minor", "value": 0 }
    ]
  },
  "validity": {
    "not_before": "2026-08-02T16:00:00Z",
    "achieve_by": "2026-08-15T23:59:59Z",
    "verify_by": "2026-08-17T23:59:59Z",
    "sustain_for": "P1D"
  },
  "constraints": [
    {
      "id": "no-open-dispute",
      "scope": "interval",
      "predicate": { "op": "eq", "path": "/dispute/open", "value": false }
    }
  ],
  "verification": {
    "policy": {
      "id": "urn:tsto:verification:ar.invoice-settlement:v1",
      "digest": "sha-256:CCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCCC"
    },
    "parameters": {
      "allowed_evidence_kinds": ["bank_settlement_receipt", "erp_ledger_entry"],
      "max_clock_skew_seconds": 300
    }
  },
  "integrity": {
    "alg": "sha-256",
    "canonicalization": "RFC8785",
    "value": "sha-256:DDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDDD"
  }
}
```

上例是信息性示例；profile.digest、证据摘要、策略摘要和 integrity.value 使用格式占位值，不构成加密测试向量。

## 7.2 spec

- REQUIRED。
- 值 MUST 精确等于 TSTO/00。
- 不得依据模糊的向前兼容假设解释其他版本。

## 7.3 id

- REQUIRED。
- MUST 是绝对 URI。
- MUST 在其发行命名域内全局唯一。
- 推荐使用符合 RFC 9562 的 UUID URN。
- 同一 id 出现不同内容摘要时，实现 MUST 报告身份冲突并拒绝自动覆盖。

## 7.4 type

- REQUIRED。
- 值 MUST 精确等于 TargetStateTransition。

## 7.5 issued\_at

- REQUIRED。
- MUST 是 RFC 3339 date-time。
- SHOULD 使用 UTC 的 Z 表示。
- 表示该对象被固定为不可变内容的时间，不代表接受、授权或开始执行的时间。

## 7.6 profile

- REQUIRED。
- 是一个不可变资源引用，包含 id 与 digest。
- Profile MUST 定义 Subject 的 State Projection、路径的数据类型、证据映射、允许的谓词操作符以及领域验证要求。
- Profile MUST 带版本，且相同 id + digest 的语义不得改变。
- Profile 不得改变任何 Core 字段的含义。

## 7.7 subject

- REQUIRED。
- id MUST 是绝对 URI，并在 Profile 定义的作用域内唯一标识业务对象。
- type MUST 是非空字符串，其含义由 Profile 定义。
- version OPTIONAL，用于绑定源系统版本、ETag、账本高度或快照版本。
- Subject 标识符 SHOULD 避免直接包含姓名、手机号、证件号等个人信息。

## 7.8 baseline

- REQUIRED。
- `observed_at` MUST 是 RFC 3339 时间。
- `claims` MUST 至少包含一个谓词，所有顶层谓词按逻辑 AND 解释。
- `evidence` MUST 至少包含一个 Evidence Reference。
- `observed_at` 不得晚于 `issued_at`；它描述的是用于固定 Baseline 的业务状态观察时间，而不是证据文件的上传时间。
- Baseline 的每个 claim MUST 能被至少一项 Evidence 支持，具体映射由 Profile 或 Verification Policy 定义。
- 在形成 Delegation 前，使用方 SHOULD 根据 Profile 的新鲜度规则重新确认 Baseline；过期或冲突证据不得被默认为有效。

## 7.9 target

- REQUIRED。
- `claims` MUST 至少包含一个谓词，所有顶层谓词按逻辑 AND 解释。
- Target MUST 可以针对 Profile 定义的 State Projection 进行确定性求值。
- Target 不表达执行方法。多个执行方法只要产生同一可验证终态，都可以满足同一 TSTO。

## 7.10 validity

- REQUIRED。
- `not_before` OPTIONAL；省略时取 `issued_at`。
- `achieve_by` REQUIRED；目标证据的观察时间不得晚于该时间。
- `verify_by` REQUIRED；最终验证必须在该时间前形成。
- `sustain_for` OPTIONAL；表示 Target 必须连续成立的最短时长。省略时表示只要求在有效目标观察点成立。
- 必须满足  $\text{issued\_at} \leq \text{not\_before} \leq \text{achieve\_by} \leq \text{verify\_by}$ 。
- 如果设置 `sustain_for`，Verification Policy MUST 证明 Target 在 `achieve_by` 之前首次成立，并在其后至少持续该时长；`verify_by` 必须足以覆盖该持续期。

## 7.11 constraints

- REQUIRED，可以是空数组。
- 每项约束包含本地唯一 `id`、`scope` 和 `predicate`。
- `scope = at_target` 表示只在目标验证状态求值。
- `scope = interval` 表示从 `not_before`（缺省时为 `issued_at`）到最后一个用于证明 Target 的观察时间期间必须成立。
- 任一约束被证明违反时，TSTO 不得判定为 SATISFIED。
- 如果 `interval` 约束缺乏足够的连续或采样证据，结果 MUST 为 INDETERMINATE，不得假定约束成立。

## 7.12 verification

- REQUIRED。
- policy MUST 是包含 id 和 digest 的不可变资源引用。
- parameters OPTIONAL，且必须是 JSON 对象；其字段由对应 Policy 定义。
- Verification Policy MUST 至少定义：
  1. 可接受证据种类与来源要求；
  2. 证据新鲜度、时间源与允许的时钟偏差；
  3. State Projection 的生成方法；
  4. Target 和 Constraints 的求值方法；
  5. 验证主体资格、独立性或法定来源要求（如适用）；
  6. 冲突证据、撤销、回滚和状态逆转的处理；
  7. 三值结论的形成规则与最终性条件。
- Policy 不得将“执行者自称完成”直接等同于“已验证满足”，除非 Profile 明确允许并披露这种信任模型。

## 7.13 integrity

- REQUIRED。
- Draft 00 只允许 alg = sha-256 和 canonicalization = RFC8785。
- value 格式为 sha-256:<base64url-no-padding>。
- 摘要计算步骤：
  1. 复制完整 TSTO；
  2. 删除顶层 integrity 成员；
  3. 按 RFC 8785 对剩余 JSON 做规范化；
  4. 对所得 UTF-8 字节计算 SHA-256；
  5. 以无填充 Base64URL 编码，并加 sha-256：前缀。
- 签名不是 TSTO Core 字段。外部责任事件、证据对象或签名信封 SHOULD 将 id 与 integrity.value 同时纳入签名覆盖范围或可信时间证明；不得将本句解释为对两个字符串进行未定义的直接拼接。

## 7.14 草稿与正式发行

业务产品 MAY 允许用户先保存尚未具备 Baseline、Evidence、Profile、Verification Policy 或摘要的目标草稿，但该草稿不是符合 TSTO/00 的对象。

一个对象只有同时满足以下条件，才能被声明为 Issued TSTO：

1. 包含全部 REQUIRED 字段；
2. 通过 JSON Schema 和本规范的语义校验；
3. Baseline 已绑定可解析或可授权取得的 Evidence；

4. Profile 与 Verification Policy 已固定版本和摘要；

5. integrity.value 已正确计算。

未正式发行的草稿 MUST NOT 被宣称为 TSTO/00 compliant，也不得作为可验证完成、自动付款或跨主体责任分配的最终依据。正式发行后，任何修改都必须产生新的 TSTO。

## 8. 最小谓词语言 TSTO-Predicate/00

### 8.1 路径

path 使用 RFC 6901 JSON Pointer，指向 Profile 生成的 State Projection。路径不得直接解释源系统私有结构；源系统字段必须先由 Profile 规范化。

### 8.2 比较操作

| op       | 必需成员        | 语义                        |
|----------|-------------|---------------------------|
| eq       | path, value | 观察值与 value 类型和值均相等。       |
| ne       | path, value | 不相等。                      |
| lt / lte | path, value | 小于 / 小于等于。                |
| gt / gte | path, value | 大于 / 大于等于。                |
| in       | path, value | 观察值属于 value 数组。           |
| contains | path, value | 观察数组含该元素，或观察字符串含该字符串。     |
| exists   | path, value | value 为布尔值，表示该路径应存在或不应存在。 |

Profile MUST 限定每条路径可用的操作符和数据类型。涉及金额或高精度数值时，Profile SHOULD 使用最小货币单位整数；若使用十进制字符串，必须同时定义确定性的比较规则。

### 8.3 逻辑操作

```
{ "op": "all", "args": [<predicate>, <predicate>] }
{ "op": "any", "args": [<predicate>, <predicate>] }
{ "op": "not", "arg": <predicate> }
```

all 和 any 的 args 不得为空。谓词求值不得执行代码、访问网络或产生副作用；外部数据必须先进入 State Projection 或 Evidence。

## 9. Evidence Reference

Evidence Reference 包含：

| 字段          | 要求       | 含义                                   |
|-------------|----------|--------------------------------------|
| id          | REQUIRED | 证据的绝对 URI。                           |
| kind        | REQUIRED | 由 Profile 或 Policy 定义的证据类别。          |
| digest      | REQUIRED | sha-256:<base64url-no-padding> 内容摘要。 |
| observed_at | REQUIRED | 该证据观察或记录业务状态的时间。                     |
| media_type  | OPTIONAL | 证据内容的媒体类型。                           |

| 字段       | 要求       | 含义                |
|----------|----------|-------------------|
| location | OPTIONAL | 经授权后可解析证据的绝对 URI。 |

证据内容可以保持私有。交换方可以只交换引用和摘要，并通过受控接口向获授权的 Verifier 提供原文。能解析引用不等于拥有读取权限。

## 10. 三值验证语义

Verifier 对一个 TSTO 只能形成以下语义结论：

| 结论            | 条件                                                       |
|---------------|----------------------------------------------------------|
| SATISFIED     | Target 全部为真；Constraints 未被违反；时间条件满足；Policy 要求的证据和最终性均满足。 |
| NOT_SATISFIED | 有充分证据证明 Target 不成立、时间窗口已关闭且目标未达成，或 Constraint 被违反。       |
| INDETERMINATE | 证据缺失、冲突、不可访问、不可验证，或 Policy 无法形成确定结论。                     |

INDETERMINATE 不得自动转化为 NOT\_SATISFIED，除非对应 Policy 明确规定“未能在期限内提交指定证据”本身构成不满足。

“已终止”不是 TSTO 验证结论；它是外部责任或控制事件。“已付款”也不是责任归因；它只是某类 Profile 下可能满足 Target 的状态事实。

## 11. 不可变性、修订与组合

- TSTO 一经发行 MUST 不可变。
- 修改 Subject、Baseline、Target、Validity、Constraints、Profile 或 Verification Policy 时，MUST 创建新的 id 和新的摘要。
- 新旧对象的 supersedes、derived\_from、part\_of 等关系由外层注册表或事件系统表达，Draft 00 不将这些关系放入 Core。
- Draft 00 的一个 TSTO MUST 只有一个 Subject。
- 多对象结果、阶段性结果、可选结果和组合交易 SHOULD 拆成多个原子 TSTO，由外层组合对象组织。
- 部分完成不是 Core 结论。需要按阶段计价时，应为每个可独立验证和结算的里程碑发行独立 TSTO。

## 12. 外部协议与 Binding

TSTO/00 独立于任何责任、身份、工作流、Agent、市场或结算协议。外部系统 MAY 使用 TSTO，但不得要求 TSTO Core 承载其专属字段。

外部事件、任务或交易引用 TSTO 时，MUST 同时绑定对象 id 和内容摘要：

```
{
  "object_ref": {
    "type": "TargetStateTransition",
    "spec": "TSTO/00",
    "id": "urn:uuid:01987654-3210-7abc-8def-0123456789ab",
    "digest": "sha-256:<tsto-integrity-value>"
  }
}
```

Binding MUST 遵守以下规则：

1. 不得修改 TSTO Core 字段的含义；
2. 不得只引用 id 而忽略摘要；
3. 责任、授权、执行、付款和终止状态不得回写到原 TSTO；
4. 一次实际验证事件必须区分于 TSTO 中预先指定的 `verification.policy`；
5. Binding 专属字段必须放在外部对象中，而不是扩张 TSTO Core；
6. 未使用任何 Binding 的 TSTO 仍然可以符合 TSTO/00。

如果外部协议对引用对象进行签名，TSTO 的 `id` 与 `digest` MUST 同时位于该协议定义的签名覆盖范围内。仅对 `id`、仅对可变解析地址或仅对未声明编码的摘要进行签名，不构成对特定 TSTO 内容的绑定。

责任、执行、验证和结算可以分别使用不同 Binding；采用其中一个不得被解释为必须采用其他 Binding。

### 13. Profile 合规要求

一个 TSTO/00 Profile 至少 MUST 发布：

1. 带版本的 Profile ID 和内容摘要；
2. Subject 类型与标识规则；
3. State Projection 的 JSON Schema；
4. 每条路径的数据类型、单位与枚举；
5. 源数据到 State Projection 的确定性映射；
6. 允许的谓词操作符；
7. Baseline 证据种类和新鲜度规则；
8. Target 证据种类与状态最终性；
9. PII、商业秘密和访问控制要求；
10. 至少一个有效例、一个无效例和一个 INDETERMINATE 例。

建议 Draft 00 配套试验三个 Profile：

- `commerce.order-recovery.v1`：未完成订单恢复或售后闭环；
- `ar.invoice-settlement.v1`：逾期应收到账；
- `freight.claim-paid.v1`：货运理赔到账。

只有当三个 Profile 可以共享同一 Core，才说明窄腰足够稳定。

### 14. 合规等级

| 等级            | 要求                                                                                      |
|---------------|-----------------------------------------------------------------------------------------|
| TSTO Producer | 通过 JSON Schema；满足本规范语义约束；正确计算摘要。                                                        |
| TSTO Resolver | 按 <code>id + digest</code> 解析对象；检测冲突；校验摘要、Profile 和 Policy。                             |
| TSTO Verifier | 生成确定性 State Projection；执行 Policy；输出三值结论和证据引用。                                           |
| TSTO Binding  | 外部协议同时绑定 TSTO <code>type</code> 、 <code>spec</code> 、 <code>id</code> 与摘要，且不改变 Core 语义。 |

声称“TSTO/00 compliant”的实现 MUST 明确声明其满足的等级和 Profile，不得只声明模糊的总体合规。

## 15. 安全与隐私考虑

实现至少必须处理：

- **篡改与对象替换**：所有引用同时绑定 id 和摘要；
- **陈旧基线**：Profile 规定证据新鲜度，在正式接受或开始执行前重新确认；
- **重放**：证据和事件使用唯一标识、观察时间、必要时使用 nonce 或可信时间服务；
- **时钟分歧**：Policy 指定可信时间源和最大允许偏差；
- **验证者串谋**：Policy 指定独立性、法定来源、法定人数或交叉验证；
- **选择性披露**：只暴露判断所需的最小 State Projection 与证据引用；
- **标识符泄露**：避免在 URI 中嵌入直接个人信息；
- **状态逆转**：Policy 定义撤销、回滚、退款、拒付等对最终性的影响；
- **解析攻击**：限制 JSON 深度、谓词节点数量、字符串长度和外部资源大小；
- **未知语义**：未知 Core 字段、未知操作符、无法取得的 Profile 或 Policy 必须导致拒绝或 INDETERMINATE，不得猜测。

TSTO 的内容摘要只证明内容一致性，不证明发行者身份、数据真实性、授权有效性或法律效力。

## 16. 互操作与升级门槛

Draft 00 升级为 Draft 01 前，建议至少满足：

1. 三个不同领域 Profile 无需修改 Core；
2. 两个相互独立的 Producer/Resolver 实现可以互操作；
3. 同一测试向量的摘要与谓词求值结果完全一致；
4. 至少一个独立 Binding 实现完成责任或执行事件、实际验证和结果记录的全链路；
5. 实际案例覆盖证据冲突、超时、终止、回滚和 INDETERMINATE；
6. 有外部团队提出明确的互操作需求。

在这些条件满足前，Draft 00 可以补充说明和修复错误，但不应为了单一客户把行业字段加入 Core。

## 17. 命名决议

### 17.1 Draft 00 正式名称

本草案建议采用：

- **规范/发布名**：TSTO/00
- **英文全称**：Target State Transition Object Specification
- **对象简称**：TSTO
- **对象类型**：TargetStateTransition

## 17.2 为什么使用 TSTO 而不是 TST Protocol

TST 能准确缩写 Target State Transition，但 RFC 3161 已使用 TST 表示 TimeStampToken，RFC 2756 的 HTCP 也定义过 TST 操作码。目标状态对象未来会频繁进入签名、时间戳和证据系统，直接复用 TST 容易在同一上下文产生歧义。

Draft 00 因此采用 TSTO：

- TST 保留 Target State Transition 的语义；
- 0 明确它是 Object，而不是传输协议或执行协议；
- 正式文档编号、Schema ID 和类型引用统一使用 TSTO/00。

名称检索只能降低冲突风险，不能替代正式的商标、域名和标准注册审查。进入 Draft 01 前 SHOULD 再完成一次正式命名审查。

## 18. Draft 00 决议摘要

| 问题              | 决议                                                   |
|-----------------|------------------------------------------------------|
| 交易的是 TSTO 本身吗？  | 不是。交易的是对产生该变化的承诺、责任或服务；TSTO 是被引用的目标对象。               |
| TSTO 是否包含执行步骤？  | 不包含。                                                 |
| TSTO 是否包含责任人？   | 不包含，由外部责任协议或 Binding 表达。                             |
| TSTO 是否包含价格和结算？ | 不包含，由 Market/Settlement Profile 表达。                  |
| 目标草稿是否就是 TSTO？  | 不是；只有绑定 Baseline、Evidence、Profile、Policy 和摘要后才可正式发行。 |
| TSTO 是否可变？      | 不可变；修改即发行新对象。                                        |
| 完成是否二值？         | 验证使用三值语义：满足、不满足、无法确定。                                |
| 行业差异放在哪里？       | Profile 和 Verification Policy。                       |
| 正式简称是什么？        | 发布名 TSTO/00；对象简称 TSTO。                               |

## 附录 A：TSTO/00 JSON Schema

以下 Schema 使用 JSON Schema Draft 2020-12。结构校验通过不代表语义合规；时间顺序、摘要正确性、Profile 约束和谓词类型仍需语义验证器检查。

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "urn:tsto:schema:00",
  "title": "TSTO/00 Target State Transition Object",
  "type": "object",
  "additionalProperties": false,
  "required": [
    "spec",
    "id",
    "type",
    "issued_at",
    "profile",
    "subject",
    "baseline",
    "target",
    "validity",
    "constraints",
    "verification",
    "integrity"
  ],
}
```

```

"properties": {
  "spec": { "const": "TSTO/00" },
  "id": { "type": "string", "format": "uri" },
  "type": { "const": "TargetStateTransition" },
  "issued_at": { "type": "string", "format": "date-time" },
  "profile": { "$ref": "#/$defs/immutableResourceRef" },
  "subject": { "$ref": "#/$defs/subject" },
  "baseline": { "$ref": "#/$defs/baseline" },
  "target": { "$ref": "#/$defs/target" },
  "validity": { "$ref": "#/$defs/validity" },
  "constraints": {
    "type": "array",
    "items": { "$ref": "#/$defs/constraint" }
  },
  "verification": { "$ref": "#/$defs/verification" },
  "integrity": { "$ref": "#/$defs/integrity" }
},
"$defs": {
  "digest": {
    "type": "string",
    "pattern": "^sha-256:[A-Za-z0-9_-]{43}$"
  },
  "immutableResourceRef": {
    "type": "object",
    "additionalProperties": false,
    "required": ["id", "digest"],
    "properties": {
      "id": { "type": "string", "format": "uri" },
      "digest": { "$ref": "#/$defs/digest" }
    }
  },
  "subject": {
    "type": "object",
    "additionalProperties": false,
    "required": ["id", "type"],
    "properties": {
      "id": { "type": "string", "format": "uri" },
      "type": { "type": "string", "minLength": 1 },
      "version": { "type": "string", "minLength": 1 }
    }
  },
  "evidenceRef": {
    "type": "object",
    "additionalProperties": false,
    "required": ["id", "kind", "digest", "observed_at"],
    "properties": {
      "id": { "type": "string", "format": "uri" },
      "kind": { "type": "string", "minLength": 1 },
      "digest": { "$ref": "#/$defs/digest" },
      "observed_at": { "type": "string", "format": "date-time" },
      "media_type": { "type": "string", "minLength": 1 },
      "location": { "type": "string", "format": "uri" }
    }
  },
  "pointer": {
    "type": "string",
    "pattern": "^(?:(?:[~/]|[~01])*)+$"
  },
  "comparisonPredicate": {
    "type": "object",
    "additionalProperties": false,
    "required": ["op", "path", "value"],
    "properties": {
      "op": {
        "enum": ["eq", "ne", "lt", "lte", "gt", "gte", "in", "contains"]
      },
      "path": { "$ref": "#/$defs/pointer" },
      "value": true
    }
  },
  "existsPredicate": {
    "type": "object",
    "additionalProperties": false,
    "required": ["op", "path", "value"],
    "properties": {
      "op": { "const": "exists" },
      "path": { "$ref": "#/$defs/pointer" },
      "value": { "type": "boolean" }
    }
  }
}

```

```

    },
    "listPredicate": {
      "type": "object",
      "additionalProperties": false,
      "required": ["op", "args"],
      "properties": {
        "op": { "enum": ["all", "any"] },
        "args": {
          "type": "array",
          "minItems": 1,
          "items": { "$ref": "#/$defs/predicate" }
        }
      }
    },
    "notPredicate": {
      "type": "object",
      "additionalProperties": false,
      "required": ["op", "arg"],
      "properties": {
        "op": { "const": "not" },
        "arg": { "$ref": "#/$defs/predicate" }
      }
    },
    "predicate": {
      "oneOf": [
        { "$ref": "#/$defs/comparisonPredicate" },
        { "$ref": "#/$defs/existsPredicate" },
        { "$ref": "#/$defs/listPredicate" },
        { "$ref": "#/$defs/notPredicate" }
      ]
    },
    "baseline": {
      "type": "object",
      "additionalProperties": false,
      "required": ["observed_at", "claims", "evidence"],
      "properties": {
        "observed_at": { "type": "string", "format": "date-time" },
        "claims": {
          "type": "array",
          "minItems": 1,
          "items": { "$ref": "#/$defs/predicate" }
        },
        "evidence": {
          "type": "array",
          "minItems": 1,
          "items": { "$ref": "#/$defs/evidenceRef" }
        }
      }
    },
    "target": {
      "type": "object",
      "additionalProperties": false,
      "required": ["claims"],
      "properties": {
        "claims": {
          "type": "array",
          "minItems": 1,
          "items": { "$ref": "#/$defs/predicate" }
        }
      }
    },
    "validity": {
      "type": "object",
      "additionalProperties": false,
      "required": ["achieve_by", "verify_by"],
      "properties": {
        "not_before": { "type": "string", "format": "date-time" },
        "achieve_by": { "type": "string", "format": "date-time" },
        "verify_by": { "type": "string", "format": "date-time" },
        "sustain_for": { "type": "string", "format": "duration" }
      }
    },
    "constraint": {
      "type": "object",
      "additionalProperties": false,
      "required": ["id", "scope", "predicate"],
      "properties": {
        "id": { "type": "string", "minLength": 1 },
        "scope": { "enum": ["at_target", "interval"] },

```

```

    "predicate": { "$ref": "#/$defs/predicate" }
  },
  "verification": {
    "type": "object",
    "additionalProperties": false,
    "required": ["policy"],
    "properties": {
      "policy": { "$ref": "#/$defs/immutableResourceRef" },
      "parameters": { "type": "object" }
    }
  },
  "integrity": {
    "type": "object",
    "additionalProperties": false,
    "required": ["alg", "canonicalization", "value"],
    "properties": {
      "alg": { "const": "sha-256" },
      "canonicalization": { "const": "RFC8785" },
      "value": { "$ref": "#/$defs/digest" }
    }
  }
}
}
}

```

## 附录 B：开放问题

Draft 01 前不应提前冻结，但必须通过实现回答：

1. Profile 和 Verification Policy 的发现、缓存和撤销机制；
2. 对持续状态的最小证据采样规则；
3. 多验证者、法定人数和冲突验证的统一表达；
4. 零知识证明、选择性披露和机密计算的证据接口；
5. 状态回滚后，历史 SATISFIED 结论如何被后续事件修正但不被删除；
6. 多 TSTO 组合、条件依赖和原子结算的上层规范；
7. TSTO Registry、Profile Registry 和测试向量的治理方式。

## 附录 C：JEP 关系与 Binding 入口（非规范性）

本附录说明 TSTO/00 与 JEP-Core 0.6 的分层关系，不构成 TSTO/00 的依赖，也不替代独立的 JEP-TSTO Binding/00。实现 TSTO 不要求实现 JEP；实现 JEP 也不要求引用 TSTO。

JEP-Core 记录围绕某个对象发生的、可验证签名的 Judgment、Delegation、Termination 或 Verification 事件声明；它不自行证明外部事实、授权有效性、法律责任或委托的可执行性。TSTO 负责表达“什么变化才算完成”。

下面的 object\_ref 只是协议中立的引用片段，不是完整 JEP 事件，也不是 JEP-Core 顶层字段：

```

{
  "object_ref": {
    "type": "TargetStateTransition",
    "spec": "TSTO/00",
    "id": "urn:uuid:01987654-3210-7abc-8def-0123456789ab",
    "digest": "sha-256:<tsto-integrity-value>"
  }
}

```

建议的语义映射如下：

| JEP 事件       | 与 TSTO 的关系                                                      |
|--------------|-----------------------------------------------------------------|
| Judgment     | 某主体记录其提出、接受、拒绝或重新评估该 TSTO 的判断声明。                                |
| Delegation   | 某主体记录其围绕该 TSTO 作出的任务、权限、责任或决策上下文委托声明；JEP 事件有效不等于外部授权有效。         |
| Termination  | 记录终止某项既有委托、权限或未来依赖；应引用被终止的 JEP 事件，不得删除或修改 TSTO。                 |
| Verification | 引用 TSTO、Verification Policy、Evidence 和三值结论，记录一次已发生且明确声明范围的验证事件。 |

TSTO 中的 `verification.policy` 是预先固定的领域事实验证规则；JEP Verification 是一次已经发生的签名事件。JEP Trust Profile 负责事件主体与签名密钥等信任绑定。三者 MUST 保持分离。

TSTO Profile 定义业务对象的 State Projection、字段、证据与谓词；JEP Profile 定义身份、密钥、凭证、授权上下文、归档或其他 JEP 互操作规则。二者名称相似，但不是同一类 Profile。

TSTO 使用 `sha-256:<base64url-no-padding>` 作为对象内容摘要；JEP-Core 0.6 使用 `sha256:<lowercase-hex>` 作为 JEP 事件摘要。Binding MUST 保留并声明各自的编码，不得将二者直接字符串比较，也不得把 TSTO 摘要冒充为 JEP event hash。

JEP-Core 的 `ref` 可以承载外部对象引用，`what` 承载事件声明。完整的 JEP-TSTO 事件 MUST 使用 JEP-Core 的真实顶层结构，并使 TSTO 的 `id` 和 `digest` 均处于 JEP 签名覆盖范围内。具体字段、三值结果映射、四类事件示例和验证规则由独立的 JEP-TSTO Binding/00 定义。

JEP-TSTO Binding 不得修改 TSTO Core 或 JEP-Core 的语义。未实现该 Binding 不影响任何一方的独立合规。

## 参考资料

- [RFC 2119 - Key words for use in RFCs to Indicate Requirement Levels](#)
- [RFC 8174 - Ambiguity of Uppercase vs Lowercase in RFC 2119 Key Words](#)
- [RFC 3339 - Date and Time on the Internet: Timestamps](#)
- [RFC 3986 - Uniform Resource Identifier: Generic Syntax](#)
- [RFC 6901 - JavaScript Object Notation \(JSON\) Pointer](#)
- [RFC 8785 - JSON Canonicalization Scheme](#)
- [RFC 9562 - Universally Unique Identifiers \(UUIDs\)](#)
- [JSON Schema Draft 2020-12](#)
- [RFC 3161 - Internet X.509 Public Key Infrastructure Time-Stamp Protocol](#)
- [RFC 2756 - Hyper Text Caching Protocol](#)
- [Judgment Event Protocol \(JEP\) - draft-wang-jep-judgment-event-protocol-06](#)
- [JEP Profiles and Interoperability - draft-wang-jep-profiles-00](#)
- [JEP Conformance and Test Suite - draft-wang-jep-conformance-00](#)